

Lecture 32 : Cryptographic hash functions (Part II)

Introduction

In an increasingly interconnected digital world, the integrity and authenticity of information are paramount. From safeguarding personal passwords to ensuring the unaltered transmission of critical data, cryptographic hash functions serve as silent guardians, providing a robust layer of security. This lecture delves deeper into the fascinating realm of these one-way mathematical operations, building upon the foundational properties discussed previously. We will explore their diverse applications, examine the intricate workings of advanced hash algorithms like SHA-512, and introduce the concept of Hash-based Message Authentication Codes (HMACs). Furthermore, we will confront the inherent vulnerabilities of hash functions by discussing various attack vectors, including the notorious Birthday Attack, and understand how modern designs mitigate these threats. Understanding these concepts is crucial for anyone involved in designing or analyzing secure digital systems, as they form the bedrock of trust in our digital interactions.

Core Concepts

Term	Definition	Significance
One-Way Hash Function	A mathematical function that transforms an input (message) into a fixed-size string of characters (hash value or digest) in a way that is computationally infeasible to reverse.	Essential for security as it prevents an attacker from reconstructing the original message from its hash, crucial for password storage and data integrity.
SHA-512	A specific member of the Secure Hash Algorithm (SHA) family, producing a 512-bit (64-byte) hash value. It is considered one of the strongest cryptographic hash functions.	Provides a high level of security due to its large digest length and complex internal structure, making collision attacks extremely difficult.
HMAC	Hash-based Message Authentication Code. A specific	Offers message authentication and integrity without relying

Term	Definition	Significance
	construction for calculating a message authentication code (MAC) involving a cryptographic hash function and a secret key.	on slower encryption algorithms, making it efficient for verifying data origin and preventing tampering.
Collision	Occurs when two different input messages produce the exact same hash value.	A critical vulnerability in hash functions; finding collisions can compromise security, allowing attackers to substitute messages while maintaining the same hash.
Birthday Attack	A type of cryptographic attack that exploits the mathematics behind the birthday problem to find collisions in hash functions more quickly than brute force.	Demonstrates that the effective security strength of a hash function against collision attacks is often half its output length, necessitating larger hash sizes.
Symmetric-Key Encryption	An encryption method where the same secret key is used for both encryption and decryption.	When combined with hash functions, it can provide both authentication and message integrity, but requires secure key distribution.
Public-Key Encryption	An encryption method using a pair of keys: a public key for encryption and a private key for decryption.	Can be used with hash functions to create digital signatures, offering non-repudiation and strong authentication, though it is computationally slower.

Detailed Analysis

1. Applications of One-Way Hash Functions

One-way hash functions are fundamental to various security applications, primarily due to their irreversible nature and sensitivity to input changes.

- **Password Storage:**

- Instead of storing user passwords in plain text, which would be a severe security risk, systems store the hash values of passwords.
- When a user attempts to log in, the system hashes the entered password and compares this new hash with the stored hash value.
- If the hashes match, authentication is successful. This prevents attackers who gain access to the password database from directly obtaining user passwords.
- **Authentication and Message Integrity using Symmetric-Key Encryption:**
 - This method combines a hash function with symmetric-key encryption to achieve both sender authentication and message integrity.
 - **Sender Side:**
 - A message (M) is first passed through a hash function (H) to generate a hash digest.
 - This hash digest is then encrypted using a shared secret symmetric key (K).
 - The original message (M) and the encrypted hash are transmitted together.
 - **Receiver Side:**
 - The receiver applies the same hash function (H) to the received message (M) to generate a new hash digest.
 - The received encrypted hash is decrypted using the same shared secret key (K) to recover the original hash digest.
 - The two hash digests are compared. If they match, it confirms:
 - **Authentication:** Only someone with the secret key K could have encrypted the hash, thus authenticating the sender.
 - **Message Integrity:** Any alteration to the message during transmission would result in a different hash digest, causing the comparison to fail.
 - **Performance Note:** While effective, symmetric-key encryption is slower than hash computation alone.
- **Authentication and Message Integrity using Public-Key Encryption (Digital Signatures):**
 - This approach leverages public-key cryptography, often referred to as a digital signature process, to provide authentication and non-repudiation.
 - **Sender Side:**
 - A message (M) is hashed to produce a digest.
 - The sender encrypts this hash digest using their private key. This encrypted hash is the digital signature.

- The message (M) and the digital signature are transmitted.
- **Receiver Side:**
 - The receiver hashes the received message (M) to generate a new hash digest.
 - The received digital signature is decrypted using the sender's public key to recover the original hash digest.
 - The two hash digests are compared. If they match, it confirms:
 - **Authentication/Non-repudiation:** Only the sender possesses their private key, so only they could have created the signature, proving the sender's identity and preventing them from denying sending the message.
 - **Message Integrity:** Any modification to the message would alter its hash, leading to a mismatch.
 - **Performance Note:** Public-key encryption is significantly slower than symmetric-key encryption, making it the slowest option among these methods.
- **Authentication and Message Integrity without Encryption (using shared secret K):**
 - This method aims to achieve authentication and integrity more efficiently by avoiding encryption altogether, relying solely on a shared secret key (K) and a hash function.
- **Sender Side:**
 - A secret value (K), known only to the sender and receiver, is appended to the message (M).
 - The hash function is applied to the concatenated string `message + K`.
 - The message (M) and the resulting hash value are transmitted. The secret K is not transmitted.
- **Receiver Side:**
 - The receiver, who also possesses the secret value (K), appends it to the received message (M).
 - The hash function is applied to `message + K`.
 - This newly computed hash is compared with the received hash value.
- **Outcome:**
 - If the hashes match, it implies both sender authentication (only someone with K could generate the correct hash) and message integrity (any message alteration would change the hash).

- **Performance Note:** This method is faster than those involving encryption because it only uses hash function computations.
- **Performance Hierarchy:**
 - Hash function computation is the fastest.
 - Symmetric-key encryption is slower than hash computation.
 - Public-key encryption is the slowest. This hierarchy influences the choice of method based on performance requirements.

2. SHA-512 Hash Function

SHA-512 is a prominent member of the Secure Hash Algorithm family, designed for robust cryptographic hashing.

- **Message Preprocessing:**
 - The input message is first padded to ensure its total bit string length is a multiple of 1024 bits.
 - Padding involves appending a single '1' bit, followed by '0' bits, and finally a 128-bit representation of the original message's length (L).
 - This padded message is then divided into 1024-bit blocks: $M_1, M_2, \dots, M_N$.
- **Iterative Hashing Process:**
 - Each 1024-bit block undergoes the SHA-512 algorithm sequentially.
 - The process starts with an initial 512-bit hash value (often a predefined constant).
 - For the first block (M_1), the algorithm processes it with the initial hash value to produce an intermediate 512-bit hash.
 - This intermediate hash then serves as the input for processing the next block (M_2), and so on.
 - This chaining continues until all blocks (M_N) have been processed.
 - The final 512-bit output after processing M_N is the message's SHA-512 hash digest.
- **The Compression Function (F):**
 - The core of SHA-512 is its compression function, denoted as F . This function takes a 1024-bit message block and a 512-bit intermediate hash value as input and produces a new 512-bit hash value.
 - **Internal Structure:** Inside the F function, there are 80 iterations (or rounds) for each 1024-bit block.

- **Buffer Management:** A 512-bit buffer is maintained and updated at every step of these 80 iterations.
- **Round Constants:** Each round uses a 64-bit value derived from the message block and a unique round constant. These round constants are generated from the cube roots of the first 80 prime numbers, adding to the complexity and security.
- **Operations:** The internal operations within each round involve complex bitwise operations, including sigma operations, majority operations, and XOR operations, all iterated 80 times to thoroughly mix the input data.
- **SHA Family Versions:**
 - The SHA family includes several versions, primarily distinguished by their digest length and internal parameters:
 - **SHA-1:** Produces a 160-bit hash.
 - **SHA-224:** Produces a 224-bit hash.
 - **SHA-256:** Produces a 256-bit hash.
 - **SHA-384:** Produces a 384-bit hash.
 - **SHA-512:** Produces a 512-bit hash.
 - **Processing Unit:** SHA-1, SHA-224, and SHA-256 process messages in 512-bit chunks, while SHA-384 and SHA-512 use 1024-bit chunks.
 - **Number of Rounds:** The number of internal rounds varies: SHA-1 uses 80 rounds; SHA-224 and SHA-256 use 64 rounds; SHA-384 and SHA-512 use 80 rounds.
 - **Maximum Message Size:** The maximum message length supported is very large. For SHA-1, SHA-224, and SHA-256, it's $2^{64} - 1$ bits. For SHA-384 and SHA-512, it's $2^{128} - 1$ bits, reflecting the number of bits reserved for the message length in the padding.

3. HMAC (Hash-based Message Authentication Code)

HMAC is a specific construction for generating a Message Authentication Code (MAC) that leverages existing cryptographic hash functions without requiring encryption.

- **Motivation:**
 - Traditional MAC algorithms often involve encryption, which is computationally slower.
 - HMAC aims to provide message authentication and integrity efficiently by relying solely on the speed of hash functions.

- It uses a secret key, similar to encryption-based MACs, but applies it within a hashing context.
- **HMAC Construction (Two-Step Hashing):**
 - Let M be the message and K be the secret key shared between sender and receiver. HMAC uses two distinct padding blocks, Q and P .
 - **Step 1 (Inner Hash):**
 - The secret key K is padded with block Q .
 - This padded key is concatenated with the message M : $(K || Q) || M$.
 - A cryptographic hash function (e.g., SHA-1, SHA-256) is applied to this concatenated block: $H((K || Q) || M)$. This produces an intermediate hash digest.
 - **Step 2 (Outer Hash):**
 - The secret key K is padded with block P .
 - This padded key is concatenated with the intermediate hash digest from Step 1: $(K || P) || H((K || Q) || M)$.
 - The hash function is applied again to this entire construction.
 - The final output is the HMAC value.
- **Simplified Explanation from Lecture:** The lecture describes a slightly simplified version where K is appended to M along with a padding block Q for the first hash, and then the result of this hash is combined with K and another padding block P for the second hash.
 - First hash: $H(M || Q || K)$ (where M is the message, Q is a short padding block, K is the secret key).
 - Second hash: $H(H(M || Q || K) || P || K)$ (where P is another short padding block).
- **Benefits:**
 - **Efficiency:** Avoids the computational overhead of encryption.
 - **Security:** The two-step hashing with a secret key and distinct padding blocks (P and Q) provides strong resistance against various attacks, including length extension attacks.
 - **Integrity and Authentication:** Ensures that the message has not been tampered with and originates from a sender who possesses the secret key.

4. Attacks on Hash Functions

Despite their robust design, cryptographic hash functions are targets for various attacks, primarily focusing on finding collisions.

- **Collision as the Main Issue:**

- The primary goal of an attacker against a hash function is to find a collision: two different input messages (X and X') that produce the same hash value ($H(X) = H(X')$).
- If a collision can be found, an attacker could potentially substitute a legitimate message with a malicious one, while the hash value remains unchanged, thus compromising integrity and authentication.

- **Birthday Attack:**

- This is a well-known attack that exploits the "birthday paradox" in probability theory.
- **Concept:** If a hash function produces an n -bit hash digest, a brute-force attack to find a collision would theoretically require 2^n attempts. However, the Birthday Attack suggests that a collision can be found with a much higher probability after generating approximately $2^{(n/2)}$ random messages and their corresponding hash values.
- **Implication:** This means the effective security strength of an n -bit hash function against collision attacks is roughly $n/2$ bits.
- **Requirement for n :** To resist practical Birthday Attacks, the hash digest length (n) must be sufficiently large. For example, n must be at least 128 bits to provide a security level equivalent to 64 bits (2^{64} operations), which is generally considered the minimum for strong collision resistance. SHA-512, with its 512-bit output, offers a much higher security level (2^{256} operations) against this attack, making it significantly more difficult to mount.
- **Other Attack Types:**
 - **Compression Function Attacks:** Attempts to find weaknesses within the internal compression function (like the 'F' function in SHA-512) that processes message blocks.
 - **Chaining Attacks:** Exploiting the iterative nature of hash functions where the output of one block's processing feeds into the next.
 - **Block Cipher Attacks:** If a hash function is constructed using a block cipher, attacks against the underlying block cipher could potentially be adapted to attack the hash function.

- While many types of attacks have been theorized or reported, their success against well-designed, modern hash functions like SHA-512 has been limited, especially when large enough secret values and robust algorithms are employed.

Key Takeaways

- [] Cryptographic hash functions are essential for security, providing one-way mapping for password storage, authentication, and message integrity.
- [] Encryption/decryption methods (symmetric-key, public-key) can be combined with hash functions for authentication and integrity, but are slower than pure hash computation.
- [] Public-key encryption, when used with hashing, enables digital signatures, offering non-repudiation but is the slowest method.
- [] An efficient alternative for authentication and integrity involves appending a shared secret key to the message before hashing, avoiding encryption.
- [] SHA-512 is a robust hash function that processes messages in 1024-bit blocks through an iterative compression function with 80 rounds, producing a 512-bit digest.
- [] The SHA family includes various versions (e.g., SHA-1, SHA-256, SHA-512) differing in digest length, block size, and number of rounds.
- [] HMAC provides message authentication and integrity using a two-step hashing process with a secret key and padding, offering efficiency by avoiding encryption.
- [] The Birthday Attack is a significant threat to hash functions, demonstrating that collisions can be found in approximately $2^{(n/2)}$ attempts for an n-bit hash.
- [] To counter the Birthday Attack, hash functions must have a sufficiently large output length, with at least 128 bits being a minimum for collision resistance.

Conclusion

This lecture has illuminated the critical role of cryptographic hash functions in securing digital communications, moving beyond their basic properties to explore their practical applications and underlying mechanisms. We've seen how these one-way functions are ingeniously employed for password storage, ensuring that even if a database is compromised, sensitive credentials remain protected. Furthermore, we examined sophisticated methods for guaranteeing both the authenticity of a sender

and the integrity of a message, whether through the use of symmetric-key encryption, the non-repudiation offered by public-key digital signatures, or the efficient, encryption-free approach using shared secret keys. The detailed exploration of SHA-512 showcased the complexity and iterative nature required to build a strong hash algorithm, while the introduction to HMAC highlighted how existing hash functions can be adapted to create efficient message authentication codes. Finally, understanding attacks like the Birthday Attack underscores the continuous arms race in cryptography and the necessity for ever-larger hash digest lengths to maintain security against evolving threats. As digital interactions become more pervasive, the principles and algorithms discussed here will continue to form the bedrock of trust and security, paving the way for future innovations in secure communication and data protection.